



Tecnológico Nacional de México
Instituto Tecnológico de la Paz

Puzzle numérico

15	2	1	12
8	5	6	11
4	9	10	7
3	14	13	

1	2	3	4
5	6	7	8
9	10	11	12
13	15	14	

Ingeniería en Sistemas Computacionales

Inteligencia Artificial

Docente:

- Lic. Karla Gallardo Ruíz

Alumnos:

- José Alberto Zapata Telles
- Luis Mauricio Romero Romero

Introducción

Este tipo de puzle numérico se puede resolver utilizando métodos lógicos con ensayo y error, parecidos a los que se desarrollan para enfrentarse a los sudokus.

Sin embargo, si se utilizan herramientas matemáticas se llega más fácilmente a la solución. En efecto se pueden solucionar con la ayuda del álgebra. Basta trabajar con ecuaciones y sistemas de un nivel básico, y sobre todo realizar la búsqueda de las soluciones de forma sistemática y ordenada, comparando las ecuaciones entre sí, viendo lo que tienen en común y simplificándolas.

Juego y Reglas

- Es éste un conocido juego de tipo solitario.
- En una cuadrícula de 3x3 se colocan los números del 1 al 8, dejando una casilla vacía, que puede ser ocupada por las piezas adyacentes.
- Se trata de desordenar las 8 piezas con número y volver a recomponer el orden.
- el juego tiene solución para cualquier distribución inicial de los 8 números.

Algoritmo seleccionado para el juego

Algoritmo elegido: **A*** (**A estrella**): Es un algoritmo es del grupo de los algoritmos de búsqueda informada, es un algoritmo de estrategia, es usado en problemas específicos de conocimiento, derivados de problemas de eso mismo. En otras palabras, es una búsqueda "Primero el mejor", el algoritmo evalúa el siguiente camino posible basado en total estimado del costo. El algoritmo siempre mira por el camino más barato (g) y el camino más posible (h) e intenta llegar hacia la meta.

Pasos para resolver el puzle numérico computacionalmente

1. Obtenga el primer estado, que es un estado inicio.
2. Obtenga todos los estados posibles en los que puede estar el puzle
3. Agregue estos nuevos estados en la lista de estado abierto.
4. Agregar estados procesados en la lista cerrada.
5. Elija el siguiente estado que tenga el costo más bajo en la lista abierta.
6. Comience repitiendo los pasos del 1 al 6 a menos que esté en el estado final, o no existen más estados por examinar (En este caso no tiene solución).
7. Una vez que tenga el estado final, retroceda al nodo inicio y esa sería la ruta óptima para encontrar la solución.

Dado que cada vez que se mueve un mosaico por un bloque, puedes decir que el costo real del movimiento (g) es 1 de mover un estado a otro.

Para el puzle numérico, el total de estados en el tablero podría ser de $9!/2 = 181,440$. Entonces realmente necesitas una forma óptima de determinar si estás alcanzando la meta y sin explorar estados innecesarios (si se superan los 181,440, computacionalmente se dice que no tiene solución).

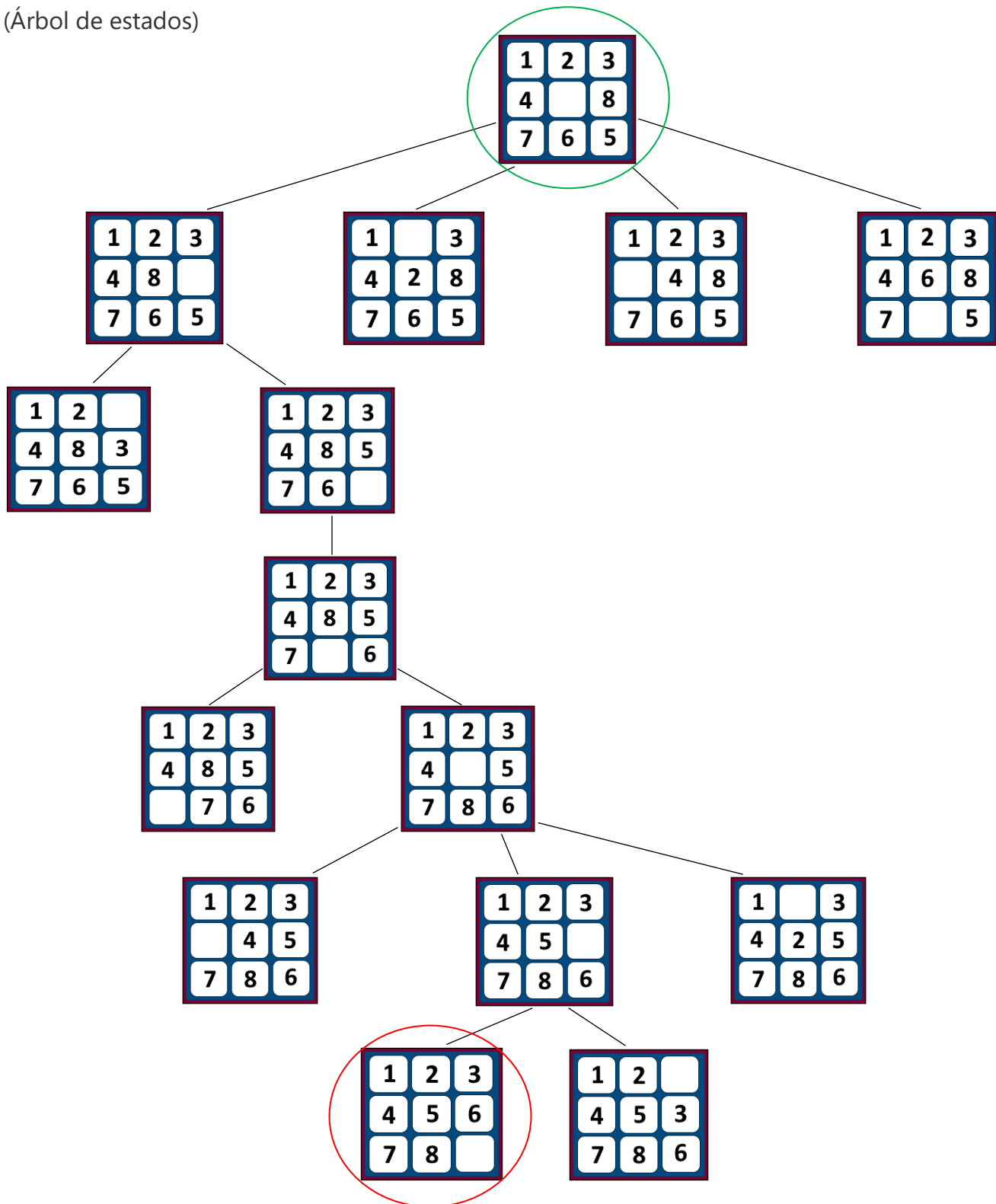
La manera de sacar el costo h es: con la suma de distancias de las fichas desde sus posiciones reales (distancia de Manhattan) - suma de distancias horizontales y verticales, para cada ficha.

Implementación en el código

- Debe mantener la información principal para mantener el árbol y poder rastrearlo.
- Además, debe mantener una lista de nodos abiertos y cerrados para no examinar el que se ha examinado anteriormente y evitar caer en el ciclo.
- Probablemente se desee mantener los nodos abiertos utilizando la estructura de datos Min Priority Queue (cola de menor prioridad) para que pueda ver el siguiente estado rápidamente. También se puede hacer sin Min Priority Queue, pero eso sería demasiado lento.
- También necesita tablas Hash para encontrar rápidamente nodos cerrados / abiertos.

Secuencia de estados

(Árbol de estados)



Rendimiento

El problema de 8-Puzzle consiste, en tablero de 3 X 3, el cual contiene 8 bloques cuadrados numerados (del 1 al 8) y un cuadrado vacío. Los cuadrados adyacentes al espacio vacío pueden moverse a ese espacio. El objetivo es alcanzar un estado meta. Una formulación posible del problema es la siguiente:

Estados: Un estado indica la posición de cada uno de los nueve cuadrados dentro del tablero.

Estado Inicial: Un estado cualquiera.

Función para obtener sucesores: A partir de un estado se generan los estados válidos que resultan de los cuatros posibles acciones. Las acciones posibles son: mover el cuadrado a la izquierda, derecha, arriba y abajo.

Costo del camino: Cada movimiento o paso, tiene un costo de uno (1), por lo tanto, el camino desde un estado inicial a un estado meta es igual al número de pasos (o movimientos) que se dan para alcanzar el estado meta.

Función de prueba de meta: Determina si un estado es el estado meta.

En la figura 2, se puede observar la arborescencia obtenida al resolver el 8-Puzzle con el algoritmo de Búsqueda en amplitud. Nuestro objetivo es encontrar el camino más corto, el cual en la figura 2 se muestra resaltado. Note que el grafo para este problema es implícito. Es decir, dado un estado inicial se obtienen los estados sucesores con una función. No se guarda todo el grafo, que es todas las posibles combinaciones del tablero, en memoria.

Se desea que resolver el 8-Puzzle con el algoritmo informado conocido como A*.

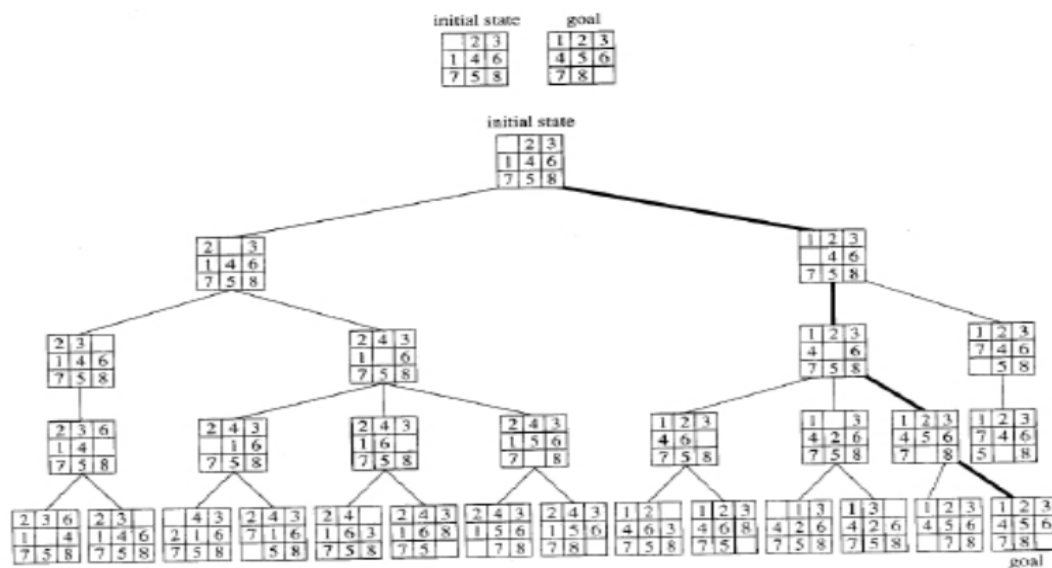


Figura 2: Arborescencia del 8-Puzzle obtenida con BFS

1. **Zero heurística:** En esta heurística todo estado da como resultado zero. Es decir, para cualquier estado n , se tiene que $h(n) = 0$
2. **Distancia Manhattan:** Es la suma de las distancias, vérticales y horizontales, de cada uno de los bloques, a su posición en el estado meta.
3. **Número de cuadros desordenados:** Es el número de bloques en posiciones equivocadas, es decir, diferentes a las que corresponden al estado meta. Se puede interpretar que mientras más desordenado esté el tablero, más difícil debe ser ordenarlo.
4. **Blanco:** Distancia entre la posición en el estado meta del blanco y su posición actual.

Por ejemplo, dado los estados inicial y final de la figura 1, se tiene que el cálculo de las heurísticas es como sigue:

- **Zero heurística:** 0

- **Distancia Manhattan:**

Cuadro	1	2	3	4	5	6	7	8
Distancia	1	2	0	0	2	2	0	3

$$\text{total} = 1 + 2 + 2 + 2 + 3 = 10$$

- **Número de cuadros desordenados:**

Cuadro	1	2	3	4	5	6	7	8
Posiciones	1	1	0	0	1	1	0	1

$$\text{total} = 1 + 1 + 1 + 1 + 1 = 5$$

- **Blanco:** 2

8	1	3
4		2
7	6	5

1	2	3
4	5	6
7	8	

Figura 1: Ejemplo de Estado Inicial y Estado Final del 8-Puzzle

Al implementar A* se debe notar que es posible encolar, o colocar en la lista de estados abiertos, el mismo estado o tablero varias veces. Para prevenir la exploración innecesaria de estados, cuando se considere los estados sucesores de un estado actual, se debe evitar encolar o generar un estado que sea igual que el estado antecesor al estado actual. En la figura 3 se muestra un ejemplo de un estado que no debe ser abierto o encolado.

8	1	3
4		2
7	6	5

8	1	3
4	2	
7	6	5

8	1	3
4		2
7	6	5

Figura 3: Estado antecesor, Estado Actual y Estado Sucesor no permitido

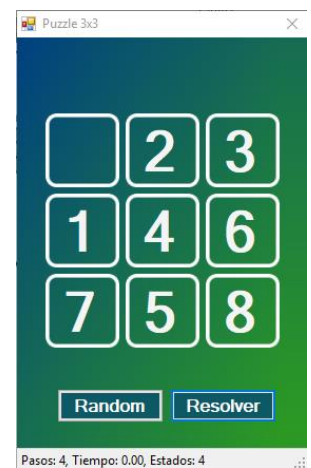
Desde cualquier tablero inicial no siempre es posible alcanzar el estado meta. El programa debe detectar si un tablero se puede resolver o no. ¡El número de tableros posibles en el 8-Puzzle es igual al número de permutaciones de 9 elementos, es decir, $9! = 362,880$. Se tiene que 8-Puzzle, tiene la propiedad de que está compuesto por 2 grafos desconectados, cada uno conteniendo $9!/2 = 181440$ estados. Por lo que se hace imperativo tener una forma eficiente de detectar si un estado se puede resolver o no.

La solución con el sistema del problema de la figura 1 es la siguiente:



Se tiene el estado inicial del problema y presionamos el botón de "Resolver" para que se empiece a buscar una solución.

Primero analiza el problema y busca una solución, son dos las posibles respuestas que dará el programa "Solución encontrada" o "Solución no encontrada". Cuando te da una solución positiva en la barra de estado nos muestra los pasos que necesita para resolver el sistema, después el tiempo que tardo en resolver el problema y el número de estado que se generaron.



Una vez que el sistema te informa que encontró solución procede a resolver el problema paso a paso, hasta llegar a estado final que es el que se muestra en la imagen. Después de encontrar la solución se pueden crear un problema arrastrando los números en la posición que se desee o bien crear un problema aleatorio.

Como se observa en la barra de estado, no tardo más de 1 segundo para encontrar la solución a este problema, se tiene que recordar que no siempre es así ya que existen muchos problemas que no tienen solución computacionalmente.